



FOR SOFTWARE ENGINEERS: THE MIGRATION YOU CAN ACTUALLY DRAW

EJB to Spring Boot

Interface compatibility is not semantic compatibility

The method signature is identical byte for byte and the contract has still changed. Two runs, one rollback, and a ledger that disagrees with itself.

Everyone agrees the old enterprise beans have to go. Nobody signs the eighteen month rewrite, and they are right not to, because nobody in the room can describe what the system looks like in month nine.

This sheet is the middle. What to extract first, what to freeze, what to measure before you touch anything, and the one failure that survives a green test suite because every type still lines up.

Every statement about a specification on this sheet was read out of that specification in September 2026 and is quoted rather than paraphrased. Statements about your own runtime are not on this page, because the only honest source for those is the version you are actually running.

In scope

- What a remote enterprise bean call carried that an HTTP call does not
- Characterisation before architecture, and what it caught here
- The anti-corruption layer, and which side of it a legacy quirk belongs on
- Four architecture rules that fail a build rather than a review
- The order of work for a strangler migration that can be stopped at any point

Out of scope

- Any claim that enterprise beans are unsupported. Jakarta EE is alive; the problem is always a specific runtime and version
- Deployment frequency numbers. Nobody has a source for those
- Which application server to leave, which is a decision only your operations record can make
- Anything implying this ordering comes from one person and one company

A circular logo featuring a stylized cartoon character with glasses and a yellow background.

Read the transaction section first. Everything else on this page is a consequence of the fact that the compiler cannot see the difference between the two call shapes.

01 The order of work, and what each step buys

1. Corpus	Record real request shapes off the running system. Not invented ones
2. Characterise	Run the corpus through the old implementation. Whatever it does IS the spec now
3. Extract the domain	Pure logic, no framework, no legacy types reaching in
4. Adapter	New domain behind the old interface. The corpus must still pass
5. Enforce	Architecture tests, so the boundary survives the next developer in a hurry
6. Divert	Route a slice of traffic. Keep the old path warm and reversible
7. Delete	Only once nothing calls it. Deleting early is how a migration becomes a rewrite

Every step leaves the system shippable. That is the property that makes the plan signable, and it is the reason characterisation comes before any architecture at all.

03 The call that carried a transaction, and the call that does not

A remote enterprise bean call made inside a transaction ran inside that transaction. The container did it for you and no line of application code said so.

Replace that call with an HTTP hop to a new service and the signature is unchanged, the arguments are unchanged, the return value is unchanged, and the transaction is gone. The caller rolls back and the work on the other side stays committed.

Nothing throws. Nothing logs. The types line up. That is what makes it dangerous: every check a reviewer would run still passes.

If a client invokes an enterprise bean method while the client is associated with a transaction context, the container invokes the enterprise bean method in the client transaction context.

`Jakarta Enterprise Beans 4.0, transaction context of a client call`

Transaction context propagation between application programs is provided by the underlying transaction manager implementations on the client and server machines. The transaction context format used for propagation is protocol dependent and must be negotiated between the client and server hosts.

`Jakarta Transactions 2.0, transaction context propagation`

An ordinary REST call from an HTTP client to a controller negotiates nothing, so there are no two transaction managers talking and no context to carry.

`Engineering practice, not vendor documentation`

Jakarta EE providers need not support the propagation of transactions to other threads and can reject resource definition annotations that include transaction as a propagated context.

`Jakarta Concurrency, ContextServiceDefinition`

04 The same two calls, one rolled back

Identical interface. Identical arguments. Identical return value. One difference, and it is in the database rather than in the response.

bean, then caller rolls back: the ledger is EMPTY. The call was enlisted in the caller transaction.

adapter, then caller rolls back: the ledger HAS ONE ENTRY. It was never enlisted.

Both paths returned the same result object, which is precisely why nobody notices until the reconciliation run.

Both call shapes return an equal result while leaving the database in different states after an identical rollback.

`Measured on the machine that built this sheet, not quoted`

05 Why the old contract is a serialization contract

A remote interface is pass by value. The arguments are serialized, so every type that crosses it is part of a Java serialization contract with all of its history baked in.

That is why the adapter inherits two contracts rather than one: the method signature, which the compiler checks, and the wire shape, which it does not.

It is also why a local interface type cannot be handed to a remote method. The spec forbids it, and the reason is the same one that makes this migration hard.

Arguments and results of methods in a local client view are passed by reference, whereas remote interfaces use pass-by-value semantics.

Jakarta Enterprise Beans 4.0, Core Features

If an entity instance is to be passed by value as a detached object through a remote interface, the entity class must implement the Serializable interface.

Jakarta Enterprise Beans 4.0, Core Features

A local interface type must not be passed as an argument or result of a remote interface method.

Jakarta Enterprise Beans 4.0, Core Features

06 What the corpus caught, which nobody planted

Thirteen recorded request shapes through the legacy bean and through the adapter. Twelve matched. One did not.

Policy P-1009, scheme "NOPE", excess null. Premium 19750 on both sides. Excess 25000 on both sides. The basis label came back NOPE/S from the bean and STD/S from the new domain.

The money is identical. The label is not. Any downstream report keyed on that label now sees a value it has never seen before, and it would have shipped, because the money reconciles and the types match.

This is the thesis in miniature, and it was found rather than argued. That is the whole case for characterising before you architect.

The legacy bean upper cased whatever scheme string arrived and used it in the basis label; the new domain parses an unknown scheme to a standard default.

Measured on the machine that built this sheet, not quoted

A corpus comparison caught a behaviour change that identical signatures, identical numbers and a passing suite all missed.

Measured on the machine that built this sheet, not quoted



07 Which side of the boundary a quirk belongs on

The obvious fix is to make the new domain reproduce the old label. Do not. Normalising an unknown scheme is the correct behaviour for new code, and importing the quirk into the domain defeats the entire exercise.

The fix belongs in the adapter, whose job is exactly this: to keep paying the old contract debts so the new code does not have to inherit them.

That is what an anti-corruption layer is for, and it is the difference between a migration and a rename. The domain stays clean. The adapter carries the label.

A legacy behaviour that is wrong for new code but observed by existing consumers belongs in the translating layer, not in the extracted domain.

Engineering practice, not vendor documentation

08 The rule that fails the build, not the review

The drift that actually happens is not exotic. Somebody reuses the legacy data object in the domain instead of writing a port. It compiles, it is already on the classpath, the pull request looks fine, and the boundary is gone.

Four rules, enforced in the test suite: the domain depends on neither the legacy contract nor the adapter, no framework package reaches into the domain, and the frozen legacy contract never reaches forward into the new code.

Write the reason into the rule. A developer who hits it at four in the afternoon should not have to go and find somebody to ask.

An architecture rule carrying its reason names the offending field, the offending type, and why the rule exists, in the failure message itself.

Measured on the machine that built this sheet, not quoted

A boundary that lives only in a diagram is a boundary that survives until the next deadline.

Engineering practice, not vendor documentation

09 The answer that beats "we moved to Spring Boot"

The weak answer is the destination. Everyone has the same destination and it says nothing about whether you have done this.

The stronger answer names what the old call gave you for free and what you had to make explicit to replace it. A remote bean call joined the caller transaction. An HTTP hop does not, so the work has to become idempotent, or compensating, or it has to stay on the same side of the boundary.

Then say what you measured before you moved anything. An answer with a corpus in it sounds like someone who has done a migration. An answer with a diagram in it sounds like someone who has read about one.

Naming what a platform did implicitly, and what replaced it explicitly, distinguishes an engineer who ran a migration from one who planned a rewrite.

Engineering practice, not vendor documentation

Before you move one call out

1. Was the old call made inside a transaction, and did it join one?
2. If the transaction is gone, what makes the work safe to repeat?
3. Do I have recorded real request shapes, or shapes I invented?
4. Does the corpus pass through the adapter before any traffic moves?
5. Which types cross the old remote boundary, and are they part of a serialization contract?
6. Is there a legacy quirk in the output that a downstream system reads?
7. Which side of the anti-corruption layer did I put that quirk on?
8. Is the boundary enforced by a test, or only by agreement?
9. Can this step be stopped and shipped as it is?
10. Is the old path still warm enough to route back to today?



Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of these you have actually shipped.

Jakarta and Jakarta EE are trademarks of the Eclipse Foundation. Spring and Spring Boot are trademarks of Broadcom. Java is a trademark of Oracle and/or its affiliates. This sheet is not affiliated with, endorsed by, or sponsored by any of them.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.